第 7 讲：二叉树与递归遍历

0. 本讲的核心问题

链表中的节点最多只有一个后继；二叉树中的节点最多有两个子节点：左子树和右子树。

```
    1
   / \
  2   3
 / \
4   5
```

树题的关键不是记住很多模板，而是建立一种递归视角：

> 如果左右子树的问题已经解决，当前节点怎样利用它们的结果得到整棵树的答案？

---

1. 基本术语

```
    1          ←     root
   / \
  2   3        ← 1
 / \
4   5          ←
```

- 根：最上面的节点；
- 父节点：直接连接到下方节点的节点；
- 子节点：某节点直接连接的下一层节点；
- 叶节点：没有子节点；
- 子树：以某节点为根的完整树；
- 深度：从根到该节点经过的边数或节点数，必须看题目定义；
- 高度：从该节点向下到最深叶子的距离；
- 层：处于相同深度的节点集合。

面试题对深度、高度有时采用不同起点定义。不要只背数字，应根据函数含义明确约定。

---

2. Python 中的二叉树节点

```
[python]
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

手动创建：

```
[python]
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
```

空树：

```
[python]
root = None
```

LeetCode 通常已经构造好树，并把根节点传给方法。

---

3. 为什么树天然适合递归

一棵非空二叉树由三部分组成：

```
+     +
```

左右子树本身仍然是二叉树，所以同一个函数可以继续处理它们。

递归函数必须说清：

1. 这个函数接收一棵什么树？
2. 它返回什么含义的结果？
3. 空树应该返回什么？
4. 当前节点怎样组合左右子树的结果？

不要从“代码第一行写什么”开始。先写函数契约。

---

4. 深度优先遍历 DFS

DFS 会沿一条分支深入到底，再返回处理其他分支。

三种经典顺序根据“当前节点何时处理”命名。

4.1 前序：根、左、右

```
[python]
def preorder(root):
    result = []

    def dfs(node):
        if node is None:
            return

        result.append(node.val)
        dfs(node.left)
        dfs(node.right)

    dfs(root)
    return result
```

对于：

```
    1
   / \
  2   3
 / \
4   5
```

前序结果：

```
1, 2, 4, 5, 3
```

4.2 中序：左、根、右

```
[python]
def inorder(root):
    result = []

    def dfs(node):
        if node is None:
            return

        dfs(node.left)
        result.append(node.val)
        dfs(node.right)

    dfs(root)
    return result
```

结果：

```
4, 2, 5, 1, 3
```

二叉搜索树的中序遍历会得到有序序列。

4.3 后序：左、右、根

```
[python]
def postorder(root):
    result = []

    def dfs(node):
        if node is None:
            return

        dfs(node.left)
        dfs(node.right)
        result.append(node.val)

    dfs(root)
    return result
```

结果：

```
4, 5, 2, 3, 1
```

后序适合“先得到子树结果，再计算当前节点”的问题。

---

5. 两种常见递归写法

5.1 使用外部状态收集结果

遍历题常见：

```
[python]
result = []

def dfs(node):
    if node is None:
        return
    result.append(node.val)
    dfs(node.left)
    dfs(node.right)
```

函数主要产生副作用，不依赖返回值组合。

5.2 每次递归返回一个明确结果

求高度时更自然：

```
[python]
def max_depth(root):
    if root is None:
        return 0

    left_depth = max_depth(root.left)
    right_depth = max_depth(root.right)

    return 1 + max(left_depth, right_depth)
```

函数契约：

> `max_depth(node)` 返回以 `node` 为根的树的最大节点层数。

空树深度为 0。非空树的最大深度等于较深子树的深度加 1。

这种写法像数学递推：

\[D(T)= \begin{cases} 0, & T=\varnothing\\ 1+\max(D(T_L),D(T_R)), & \text{otherwise} \end{cases} \]

---

6. 怎样选择前序还是后序

可以问：

> 当前节点的工作需要先知道子树答案吗？

- 不需要，先处理当前节点再向下：偏前序；
- 需要左右子树结果才能计算当前答案：偏后序；
- 在二叉搜索树中按顺序访问：中序。

这不是绝对规则，但能帮助设计。

---

7. 翻转二叉树

目标是交换每个节点的左右子树：

```
[python]
def invert_tree(root):
    if root is None:
        return None

    root.left, root.right = root.right, root.left
    invert_tree(root.left)
    invert_tree(root.right)

    return root
```

这里先交换再递归是前序思路。也可以先分别翻转原左右子树，再交换结果：

```
[python]
def invert_tree(root):
    if root is None:
        return None

    left_inverted = invert_tree(root.left)
```

```
        right_inverted = invert_tree(root.right)
        root.left = right_inverted
        root.right = left_inverted

        return root
```

两个版本都正确，但不要在交换后又误把旧的子树方向当成未交换状态。

---

8. 判断两棵树是否相同

函数契约：

> `is_same_tree(p, q)` 判断以 `p` 和 `q` 为根的两棵树是否结构相同且对应值相等。

```
[python]
def is_same_tree(p, q):
    if p is None and q is None:
        return True

    if p is None or q is None:
        return False

    if p.val != q.val:
        return False

    return (
        is_same_tree(p.left, q.left)
        and is_same_tree(p.right, q.right)
    )
```

边界分三种：

1. 两者都空：相同；
2. 只有一个空：不同；
3. 都非空：比较值和左右子树。

---

9. 对称二叉树

对称不是“左右子树相同”，而是镜像：

```
        ⇔
        ⇔
```

```
[python]
def is_symmetric(root):
    def is_mirror(left, right):
        if left is None and right is None:
            return True

        if left is None or right is None:
            return False

        if left.val != right.val:
            return False

        return (
            is_mirror(left.left, right.right)
            and is_mirror(left.right, right.left)
        )

    if root is None:
```

```
            return True

        return is_mirror(root.left, root.right)
```

关键在于给辅助函数定义正确的契约：它判断两棵树是否互为镜像。

---

10. 路径和：向下传递状态

问题：是否存在从根到叶子的路径，使节点和等于 `target_sum`。

可以把剩余目标向下传：

```
[python]
def has_path_sum(root, target_sum):
    if root is None:
        return False

    remaining = target_sum - root.val

    if root.left is None and root.right is None:
        return remaining == 0

    return (
        has_path_sum(root.left, remaining)
        or has_path_sum(root.right, remaining)
    )
```

为什么必须检查叶节点？

题目要求根到叶路径。如果在中间节点就令 `remaining == 0` 并返回真，可能把未到叶子的前缀误认为完整路径。

这种模式是：

> 从父节点向子节点传递“到当前为止的状态”。

---

11. 直径：同时返回局部信息并更新全局答案

二叉树直径是任意两个节点间最长路径的边数。最长路径不一定经过根。

对每个节点，经过它的最长路径为：

\[left_height+right_height \]

递归返回当前子树高度，同时更新全局最大直径：

```
[python]
def diameter_of_binary_tree(root):
    best = 0

    def height(node):
        nonlocal best

        if node is None:
            return 0

        left_height = height(node.left)
        right_height = height(node.right)

        best = max(best, left_height + right_height)

        return 1 + max(left_height, right_height)

    height(root)
```

```
        return best
```

这里要区分：

- `height` 返回给父节点的是一条向下的最长路径；
- `best` 保存任意节点处左右路径连接得到的全局最大值。

这是树形递归中很重要的设计模式。

---

12. 迭代 DFS：显式使用栈

递归使用系统调用栈。也可以自己维护栈：

```
[python]
def preorder_iterative(root):
    if root is None:
        return []

    result = []
    stack = [root]

    while stack:
        node = stack.pop()
        result.append(node.val)

        if node.right is not None:
            stack.append(node.right)
        if node.left is not None:
            stack.append(node.left)

    return result
```

为什么先压右节点?

栈后进先出。为了先处理左节点，需要让左节点后入栈。

中序和后序的迭代写法更复杂，初学阶段优先掌握递归版。

---

13. 复杂度与树高

访问所有节点的 DFS：

```
O(n)
```

递归额外空间取决于树高 `h`：

```
O(h)
```

平衡树：

```
h = O(log n)
```

完全倾斜的树：

```
1
 \
  2
   \
    3
```

此时：

```
h = O(n)
```

所以递归空间最坏为 `O(n)`。Python 中极深的树还可能触发递归深度限制。

14. 常见错误

14.1 没有定义递归函数的返回含义

写到一半不知道应该 `return node`、数字还是布尔值。应先写函数契约。

14.2 空树返回值不符合递推

- 求深度：空树返回 0；
- 判断相同：两者都空返回真；
- 路径和：空树通常返回假。

空树返回什么由函数含义决定，不是统一模板。

14.3 忘记使用递归返回值

```
[python]
max_depth(root.left)
max_depth(root.right)
```

如果不保存结果，就无法组合得到当前答案。

14.4 混淆节点数和边数

最大深度与直径可能采用不同单位。阅读题目并保持一致。

14.5 把根到节点路径误认为根到叶路径

叶节点条件是左右孩子都为空。

14.6 共享可变默认参数

不要写：

不要把函数定义写成 `def dfs(node, result=[]):`。

默认列表会在多次调用间共享。应在外层创建结果列表。

15. 本讲检查表

```
1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
```

16. 本讲总结

1. 二叉树由根、左子树和右子树递归组成。
2. 前序、中序、后序的区别是当前节点的处理位置。
3. 写递归前先定义函数契约和空树返回值。

4. 最大深度是典型的“组合左右子树返回值”。
5. 路径和是“向下传递状态”，直径是“返回局部信息并更新全局答案”。
6. DFS 时间通常为 `O(n)`，递归空间为 `O(h)`。